Pipes in the Desert
Concept of Prototype

E5 - TEAM A

Supervisor:
Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

April 12th, 2026

7.0 Concept of prototype

7.1 *Interface definition of Prototype*

7.1.1 General description

The prototype system is a console-based, character-level interactive program that simulates the behavior of the Pipes in the Desert game using the skeleton implementation prepared in the previous task. The goal of the prototype is to demonstrate the correctness of the object model, sequence diagrams, and system behavior through controlled user interactions.

All input and output operations are performed using standard input and output streams. The system does not use any graphical interface; instead, all commands and responses are textual and displayed in the console. This ensures compatibility with file-based testing, as required by the task description.

Input handling

User input is accepted either:

- directly from the keyboard (interactive mode), or
- from a predefined input file (for testing purposes)

When file-based testing is used, input commands are provided line-by-line in the same format as interactive input, ensuring that test execution is deterministic and reproducible.

The input consists of simple textual commands and parameters, such as:

- menu selections (numbers)
- element identifiers (IDs)
- confirmation answers (e.g., yes/no)

Each command corresponds to a specific use case or action defined in the previous documents (e.g., Move Player, Repair Pipe, Puncture Pipe, etc.).

Output handling

The system produces output exclusively through:

- System.out.println() statements
- structured textual messages describing:
    - method calls
    - object states
    - results of actions
    - system-level events (e.g., water flow, score updates)

The output is designed to be:

- deterministic and traceable
- suitable for comparison with expected output files
- sufficiently detailed to observe the internal behavior of the system

Most actions prints:

- the method being executed
- relevant parameters
- the resulting state changes

This allows testers to verify correctness by comparing the output logs with expected behavior.

Execution model

The prototype follows a turn-based interaction model, where:

- the system processes one player at a time
- the user selects an action from a predefined menu
- invalid actions do not end the turn
- a valid action or explicit "end turn" command advances the game

Each game cycle (tick) includes:

1. System-level updates (e.g., component generation, pump failures)
2. Water flow calculation
3. User interaction

If the user enters an invalid or rejected action, the system does not start a new game tick. Instead, the same user is prompted again within the same turn until a valid action is performed or the user explicitly ends the turn.

After a valid action or an "end turn" command, the system advances to the next user and continues with the next game tick, as described in the Run Game use case (UC-02).

Note: Water flow is calculated during each game tick and may also be recalculated after user actions that modify the network. (UC-18).

Handling of randomness

Since the system contains inherently random behaviors (e.g., pump breakdowns, component generation), these are replaced by explicit user-controlled decisions in the prototype.

Instead of using uncontrolled randomness:

- the system explicitly asks the Tester whether to trigger random events (e.g., “Generate new pipe? yes/no”)
- each decision is made interactively by the Tester
- this ensures deterministic execution of scenarios

This approach allows all test cases to be fully reproducible, which is essential for file-based testing.

Testing support

The prototype is designed to support file-based testing, where:

- input commands can be read from a file
- output can be redirected to a file
- the resulting output can be compared to expected outputs

This approach enables:

- repeatable testing
- validation of system behavior
- verification of use-case execution

7.1.2 Input language

<String> Description

A player chooses an option from the main menu Options

1 Start game
2 Instructions
3 Exit

<String> Description
A player chooses a team Options

- 1 Plumber
- 2 Saboteur

<String> Description
A player inputs a nickname (empty input defaults to Player<n>)

<String> Description
A player inputs the number of players (minimum 4)

<String> Description
A player chooses a game duration Options

- 1 -3 minutes
- 2 -5 minutes

<String> Description
Plumber in-game action menu Options

- 1 Move
- 2 Repair pipe
- 3 Repair pump
- 4 Change pump direction
- 5 Connect pipe end
- 6 Disconnect pipe end
- 7 Pick up pump from cistern
- 8 Pick up pipe from cistern
- 9 Insert pump into pipe
- 10 Place new pipe
- 0 End turn

<String> Description
Saboteur in-game action menu Options

- 1 Move
- 2 Puncture pipe
- 3 Change pump direction
- 0 End turn

<String> Description
A player enters a target element ID to move to

<String> Description
A player enters a pipe or element ID for a Plumber action (connect, disconnect, insert pump, place pipe)

<String> Description
A player enters a pipe index to set pump input or output direction

<String> Description
A player confirms or cancels an action Options

- yes Confirm action
- no Cancel, turn not consumed

<String> Description
A player responds to a random event prompt each game tick

Options

- yes, Confirm event
- no Cancel event

Prompts:

- Generate new pipe at cistern? (yes/no):
- Generate new pump at cistern? (yes/no):
- Trigger random pump breakdown? (yes/no):

7.1.3 Output language

7.1.3.1 Main menu

```
Welcome to Pipes in the Desert! <String>
```

Description: When the program is started, the user is greeted with a welcome banner and presented with the main menu showing the three available options.

7.1.3.2 Game setup

```
--- Game Setup --- <String>
How many players? (minimum 4): <String>
Player <n> name: <String>
Choose team for <n>: <String>
<name> joined the Plumbers. | <name> joined the Saboteurs. <String>
Choose game duration: <String>
```

```
--- Starting Game — <String>
```

Description: After choosing to start a game, the setup sequence collects each player's team assignment, nickname, and the game duration. The program confirms each player's team. Aborts and returns to main menu if either team has fewer than 2 players.

7.1.3.3 Instructions screen

```
GAME INSTRUCTIONS <String>
```

Description: Displays a summary of the game rules and team objectives. Shown when the player selects option 2 from the main menu.

7.1.3.4 Game tick header

```
========== GAME TICK ========== <String>
System.updateRoundEvents() <String>
System.requestComponentGeneration() <String>
Generate new pipe at cistern? (yes/no): <String>
Generate new pump at cistern? (yes/no): <String>
Trigger random pump breakdown? (yes/no): <String>
WaterFlowManager.recalculateFlow() <String>
System.getCurrentPlayer() <String>
--- <n>'s turn (<role>) --- <String>
System.nextTurn() <String>
[Scores] Plumbers: <int> | Saboteurs: <int> <String>
[Timer] Remaining: <int> <String>
System.checkEndCondition() <String>
```

Description: Displayed at the start of each game tick. Shows the current player's name and role. After the turn completes, displays the current score for both teams, the remaining time, and checks the end condition.

7.1.3.5 Network status display

```
=== Network Status === <String>
Pipe (ID:<int>) [OK | PUNCTURED | FREE END | Occupied by: <name>] <String>
Pump (ID:<int>) [OK | BROKEN] <String>
Spring (ID:<int>) <String>
Cistern (ID:<int>) [Pipes:<int> Pumps:<int>] <String>
<String>
Current position: <type> (ID: <int>) <String>
```

Description: Displayed before each player's action menu. Lists every network element with its current state and the player's current position.

7.1.3.6 Plumber in-game action menu

```
Actions: <String>
```

Description: An in-game menu presents all ten possible actions to the Plumber player plus the option to end the turn without acting.

7.1.3.7 Saboteur in-game action menu

```
Actions: <String>
```

Description: An in-game menu presents the three possible actions to the Saboteur player plus the option to end the turn without acting.

7.1.3.8 Move sub-menu

```
Available destinations: <String>
Pipe (ID: <int>) <String>
Pump (ID: <int>) <String>
Enter target element ID: <String>
```

Description: Shown when a player selects Move. Lists all unoccupied pipes and pumps as valid targets. Shared by both Plumber and Saboteur.

7.1.3.9 Change pump direction sub-menu

```
Connected pipes: <String>
<index>: Pipe <int> <String>
Select input pipe index: <String>
Select output pipe index: <String>
```

Description: Shown when a player on a pump selects Change Pump Direction. Available to both Plumber and Saboteur.

7.1.3.10 Plumber action sub-menus

```
Enter pipe ID to connect: <String>
Enter target element ID: <String>
Enter pipe ID to disconnect: <String>
Enter target pipe ID to split: <String>
Enter first connection element ID: <String>
Enter second connection element ID: <String>
```

Description: Additional prompts shown during specific Plumber actions that require selecting a network element by ID.

7.1.3.11 Game over screen

```
========== GAME OVER ========== <String>
Winner: <name> | Game ended in a draw. <String>
```

Description: Displayed when the timer expires. Announces the winning team or a draw if scores are equal.

7.1.3.12 Error and feedback messages

```
Invalid option. Please enter 1, 2, or 3. <String>
Invalid or rejected action. Please choose again. <String>
No current player available. <String>
You are not standing on a pipe. <String>
You are not standing on a pump. <String>
You are not carrying a pump. <String>
You are not carrying a pipe. <String>
Pipe is occupied, cannot disconnect. <String>
Pipe end disconnected. <String>
Element not found. <String>
Invalid move, try another! <String>
Invalid pipe selection. <String>
Invalid pipe or target. <String>
Invalid pipe ID. <String>
Invalid element IDs. <String>
Not enough connected pipes to change direction. <String>
Not enough connected pipes. <String>
No cisterns available. <String>
Turn ended without action. <String>
Invalid choice. Turn skipped. <String>
Each team must have at least 2 players. Plumbers: <int>, Saboteurs: <int>
<String>
```

```
Returning to main menu. <String>
Goodbye! <String>
```

Description: Feedback messages printed after an invalid or inapplicable action. Declining a confirmation does not consume the turn. Setup validation messages abort setup and return to the main menu.

7.1.3.13 Skeleton method trace outputs

```
System.startGame() <String>
System.initializeGame() <String>
System.updateRoundEvents() <String>
System.requestComponentGeneration() <String>
System.nextTurn() <String>
System.getCurrentPlayer() <String>
System.checkEndCondition() <String>
System.endGame() <String>
GameTimer.start() <String>
GameTimer.tick() <String>
GameTimer.stop() <String>
GameTimer.hasExpired() <String>
GameTimer.getRemainingTime() <String>
WaterFlowManager.recalculateFlow() <String>
WaterFlowManager.processSpring(spring) <String>
WaterFlowManager.registerDeliveredWater(<int>) <String>
WaterFlowManager.registerLeakedWater(<int>) <String>
ScoreBoard.addDeliveredWater(<int>) <String>
ScoreBoard.addLeakedWater(<int>) <String>
ScoreBoard.getPlumberScore() <String>
ScoreBoard.getSaboteurScore() <String>
ScoreBoard.determineWinner() <String>
[Player:<n>] moveTo() called. <String>
[Player:<n>] Moved to Pipe. <String>
[Player:<n>] Moved to Pump. <String>
[Player:<n>] moveTo() rejected <String>
[Player:<n>] startTurn() called. <String>
[Player:<n>] endTurn() called. <String>
[Plumber:<n>] repairPipe() called. <String>
[Plumber:<n>] Pipe repaired. <String>
[Plumber:<n>] repairPump() called. <String>
[Plumber:<n>] Pump repaired. <String>
[Plumber:<n>] changePumpDirection() called. <String>
[Plumber:<n>] Pump direction changed. <String>
[Plumber:<n>] connectPipeEnd() called. <String>
[Plumber:<n>] Free end of pipe connected. <String>
[Plumber:<n>] insertPump() called. <String>
[Plumber:<n>] Pump inserted into pipe. <String>
[Plumber:<n>] placeNewPipe() called. <String>
[Plumber:<n>] collectPumpFromCistern() called. <String>
[Plumber:<n>] Pump collected from cistern. carriedPump = true. <String>
[Plumber:<n>] collectPipeFromCistern() called. <String>
[Plumber:<n>] Pipe collected from cistern. carriedPipe = true. <String>
[Saboteur:<n>] puncturePipe() called. <String>
[Saboteur:<n>] changePumpDirection() called. <String>
[Pump:<int>] Created. <String>
[Pump:<int>] setDirection() called. <String>
[Pump:<int>] Direction updated successfully. <String>
[Pump:<int>] breakDown() called. <String>
[Pump:<int>] Pump is now broken. <String>
[Pump:<int>] repair() called. <String>
[Pump:<int>] Pump repaired and restored to working state. <String>
```

```
[Pipe:<int>] Created. <String>
[Pipe:<int>] puncture() called. <String>
[Pipe:<int>] Pipe is now punctured and leaking. <String>
[Pipe:<int>] repair() called. <String>
[Pipe:<int>] Pipe repaired and restored to intact state. <String>
[Pipe:<int>] disconnectEnd() called. <String>
[Pipe:<int>] One end of the pipe is now free. <String>
[Cistern:<int>] receiveWater() called. Amount: <int> <String>
[Cistern:<int>]  Water  received  successfully.  Total  stored  water:  <int>
<String>
[Cistern:<int>] generatePipe() called. <String>
[Cistern:<int>] New pipe generated. Available pipes: <int> <String>
[Cistern:<int>] generatePump() called. <String>
[Cistern:<int>] New pump generated. Available pumps: <int> <String>
[PipeNetwork] Created empty network. <String>
[PipeNetwork] Element added: <type> (id: <int>). <String>
[PipeNetwork] Connected <type> (id: <int>) <-> <type> (id: <int>). <String>
[PipeNetwork] Pump inserted successfully. <String>
```

Description: This section lists the standardized debug strings and log messages generated by the system's skeleton during execution. These trace outputs serve as high-level feedback to verify that method calls, state transitions, and object interactions (e.g., player movements, resource management, and network updates) are functioning correctly according to the game logic. Each entry represents a successful or attempted operation within the core game loop, specifically tracking the behavior of the System, Players, and Pipe Network components.

7.2 Real use-cases

Use-case name	**UC-01: Start Game**
Short textual description	The Tester initializes a new game session by providing player count, names, team assignments, and game duration. The system generates the initial pipe network and starts the timer.
Actors	Tester
Dialog, scenario	The Tester selects option 1 from the main menu. The system prompts for the number of players (minimum 4). For each player, the Tester enters a name and selects a team (1 for Plumber, 2 for Saboteur). The system confirms each assignment. The Tester then selects a game duration (1 for 3 minutes, 2 for 5 minutes). If each team has at least 2 players, the system initializes the game, generates the initial network (Spring → Pipe → Pump → Pipe → Cistern), places all players on the pump, starts the timer, and enters the game loop. If team size validation fails, the system returns to the main menu.

Use-case name	**UC-02: Run Game**

Short textual description	The system manages the active game session by executing game ticks, handling random events, recalculating water flow, and advancing player turns until the timer expires.
Actors	Tester, System
Dialog, scenario	Once the game starts, each tick begins with updateRoundEvents(). The Tester is prompted to confirm or cancel three random events: generating a new pipe (yes/no), generating a new pump (yes/no), and triggering a pump breakdown (yes/no). Water flow is then recalculated automatically. The current player's action menu is displayed and the Tester selects an action. After a valid action or end turn (option 0), the system advances to the next player, updates scores and timer, and checks the end condition.

Use-case name	**UC-03: Move Player**
Short textual description	A player (Plumber or Saboteur) moves to an adjacent pipe or pump by entering its element ID.
Actors	Tester
Dialog, scenario	The Tester selects option 1 (Move) from either the Plumber or Saboteur action menu. The system displays all available destinations, listing unoccupied pipes and pumps. The Tester enters the target element ID. If the target exists and is a pipe that is not already occupied by another player, or is a pump, the player's position is updated. The move fails without consuming the turn if the element ID is not found, the target is not a pipe or pump, or the target pipe is already occupied by another player. Note: the skeleton does not enforce adjacency — any valid pipe or pump in the network can be selected regardless of the player's current position.

Use-case name	**UC-04: Saboteur Punctures Pipe**
Short textual description	A Saboteur standing on a pipe punctures it, causing water to leak to the desert.
Actors	Tester

Dialog, scenario	The Tester selects option 2 (Puncture pipe) from the Saboteur action menu. The system checks that the Saboteur is standing on a pipe. If valid, the system prompts "Puncture the pipe? (yes/no)". If the Tester confirms with yes, the pipe state changes to PUNCTURED and water flow is recalculated. If the Tester enters no, the action is cancelled and the turn is not consumed. If the Saboteur is not standing on a pipe, the system prints "You are not standing on a pipe."

Use-case name	**UC-05: Plumber Repairs Punctured Pipe**
Short textual description	A Plumber standing on a punctured pipe repairs it, stopping water leakage.
Actors	Tester
Dialog, scenario	The Tester selects option 2 (Repair pipe) from the Plumber action menu. The system checks that the Plumber is standing on a pipe. If valid, the system prompts "Repair the pipe? (yes/no)". If the Tester confirms, the pipe state changes from PUNCTURED to OK and water flow is recalculated. If the Tester enters no, the turn is not consumed. If the Plumber is not standing on a pipe, the system prints "You are not standing on a pipe."

Use-case name	**UC-06: Plumber Repairs Broken Pump**
Short textual description	A Plumber standing on a broken pump repairs it, restoring its water transfer capability.
Actors	Tester
Dialog, scenario	The Tester selects option 3 (Repair pump) from the Plumber action menu. The system checks that the Plumber is standing on a pump. If valid, the system prompts "Repair the pump? (yes/no)". If the Tester confirms, the pump state changes from BROKEN to OK and water flow is recalculated. If the Tester enters no, the turn is not consumed. If the Plumber is not standing on a pump, the system prints "You are not standing on a pump."

Use-case name	**UC-07: Plumber Changes Pump Direction**
Short textual description	A Plumber standing on a pump changes its active input and output routing.
Actors	Tester

Dialog, scenario	The Tester selects option 4 (Change pump direction) from the Plumber action menu. The system checks that the Plumber is standing on a pump. If so, it displays the list of connected pipes with their indices. The Tester enters the index for the desired input pipe and then the desired output pipe. If both indices are valid and distinct, changePumpDirection() is called. If the pump is working, the direction is updated and water flow is recalculated. If the pump is broken, changePumpDirection() is called but the direction change is silently rejected internally — the turn is still consumed without any effect. If fewer than 2 pipes are connected, the system prints "Not enough connected pipes to change direction." If the Plumber is not on a pump, the system prints "You are not standing on a pump."

Use-case name	**UC-08: Saboteur Changes Pump Direction**
Short textual description	A Saboteur standing on a pump changes its routing to divert water away from cisterns.
Actors	Tester
Dialog, scenario	The Tester selects option 3 (Change pump direction) from the Saboteur action menu. The system checks that the Saboteur is standing on a pump and displays the connected pipes with their indices. The Tester enters the index of the desired input pipe and then the desired output pipe. If both indices are valid and distinct, changePumpDirection() is called. If the pump is working, the direction is updated and water flow is recalculated. If the pump is broken, changePumpDirection() is called but the direction change is silently rejected internally — the turn is still consumed without any effect. If fewer than 2 pipes are connected, the system prints "Not enough connected pipes." If the Saboteur is not on a pump, the system prints "You are not standing on a pump."

Use-case name	**UC-09: Plumber Disconnects a Pipe**
Short textual description	A Plumber disconnects one end of a pipe from the network, creating a free end and causing water to leak if flow reaches it.
Actors	Tester

Dialog, scenario	The Tester selects option 6 (Disconnect pipe end) from the Plumber action menu. The system prompts "Enter pipe ID to disconnect". The Tester enters the pipe ID. If the pipe is not occupied by another player, its end is disconnected (state becomes FREE END) and water flow is recalculated. The system confirms with "Pipe end disconnected." If the pipe is occupied, the system prints "Pipe is occupied, cannot disconnect." If the pipe ID is invalid, the system prints "Invalid pipe ID."

Use-case name	**UC-10: Plumber Connects a Pipe**
Short textual description	A Plumber connects a pipe end to an adjacent network element by entering the pipe ID and target element ID.
Actors	Tester
Dialog, scenario	The Tester selects option 5 (Connect pipe end) from the Plumber action menu. The system prompts "Enter pipe ID to connect" and then "Enter target element ID". The Tester enters both IDs. If the pipe element exists and the target element exists, connectPipeEnd() is called followed by network.connectElements(). The connectPipeEnd() method internally checks whether the pipe has a free end — if it does not, the connection is silently rejected inside the method, but network.connectElements() is still called and the turn is consumed regardless. If either ID is invalid, the system prints "Invalid pipe or target."

Use-case name	**UC-11: Plumber Picks Up Pump at Cistern**
Short textual description	A Plumber collects a generated pump from the first cistern in the network into their inventory for later insertion.
Actors	Tester

Dialog, scenario	The Tester selects option 7 (Pick up pump from cistern) from the Plumber action menu. The system checks whether at least one cistern exists in the network. If a cistern exists, it always selects the first cistern regardless of availability. The system prompts "Pick up pump from cistern? (yes/no)". If the Tester confirms with yes, collectPumpFromCistern() is called. The method internally checks that the Plumber is not already carrying a pump and that the cistern has an available pump. If both conditions are met, the pump is added to the Plumber's inventory (carriedPump = true) and the cistern's available pump count decreases. If the Plumber is already carrying a pump or no pump is available at the cistern, the method silently rejects the action but the turn is still consumed. If no cistern exists in the network, the system prints "No cisterns available."

Use-case name	**UC-12: Plumber Picks Up Pipe from Cistern**
Short textual description	A Plumber collects a generated pipe from the first cistern in the network into their inventory for later placement.
Actors	Tester
Dialog, scenario	The Tester selects option 8 (Pick up pipe from cistern) from the Plumber action menu. The system checks whether at least one cistern exists in the network. If a cistern exists, it always selects the first cistern regardless of availability. The system prompts "Pick up pipe from cistern? (yes/no)". If the Tester confirms with yes, collectPipeFromCistern() is called. The method internally checks that the Plumber is not already carrying a pipe and that the cistern has an available pipe. If both conditions are met, the pipe is added to the Plumber's inventory (carriedPipe = true) and the cistern's available pipe count decreases. If the Plumber is already carrying a pipe or no pipe is available at the cistern, the method silently rejects the action but the turn is still consumed. If no cistern exists in the network, the system prints "No cisterns available."

Use-case name	**UC-13: Plumber Places New Pipe**
Short textual description	A Plumber places a previously collected pipe into the network, connecting it between two existing elements.
Actors	Tester

Dialog, scenario	The Tester selects option 10 (Place new pipe) from the Plumber action menu. The system checks that the Plumber is carrying a pipe. If yes, the system prompts for the first connection element ID and then the second. If both elements exist, the pipe is added to the network connected between them and water flow is recalculated. If the Plumber has no pipe, the system prints "You are not carrying a pipe." If either element ID is invalid, the system prints "Invalid element IDs."

Use-case name	**UC-14: Plumber Inserts New Pump**
Short textual description	A Plumber inserts a collected pump into an existing pipe, splitting it into two segments with the pump connected between them.
Actors	Tester
Dialog, scenario	The Tester selects option 9 (Insert pump into pipe) from the Plumber action menu. The system checks that the Plumber is carrying a pump. If yes, the system prompts "Enter target pipe ID to split". The Tester enters the pipe ID. If valid, a new pump object is created and PipeNetwork.insertPump() is called, which removes the original pipe from the network, creates two new pipe segments reconnected to the original pipe's neighbors, adds the pump between them, and registers both segments on the pump. Water flow is recalculated after insertion. If the Plumber has no pump, the system prints "You are not carrying a pump." If the pipe ID is invalid or not a pipe, the system prints "Invalid pipe ID."

Use-case name	**UC-15: System Randomly Breaks Pump**
Short textual description	At the start of each game tick, the system offers to break a working pump to simulate random breakdown in a deterministic and controllable way.
Actors	Tester, System
Dialog, scenario	At the start of each game tick, updateRoundEvents() is called. It first calls requestComponentGeneration(), prompting the Tester for pipe and pump generation (yes/no). It then prompts "Trigger random pump breakdown? (yes/no):". If the Tester enters yes, the first working pump in the network changes to BROKEN state. After all three prompts are resolved, recalculateFlow() is called once at the end of updateRoundEvents() to account for all state changes — component generation, and pump breakdown — together. If no working pumps exist, the breakdown prompt still appears but selecting yes has no effect. If the Tester enters no, no breakdown occurs.

Use-case name	**UC-16: System Generates New Pipe**
Short textual description	At the start of each game tick, the system offers to generate a new pipe at the cistern, making it available for a Plumber to collect.
Actors	Tester, System
Dialog, scenario	At the start of each game tick, the system prints "Generate new pipe at cistern? (yes/no):". If the Tester enters yes, the cistern's available pipe count increases by one. The system confirms with the trace "[Cistern:<int>] New pipe generated. Available pipes: <int>". The pipe remains at the cistern until a Plumber picks it up using option 8. If the Tester enters no, no pipe is generated.

Use-case name	**UC-17: System Generates New Pump**
Short textual description	At the start of each game tick, the system offers to generate a new pump at the cistern, making it available for a Plumber to collect.
Actors	Tester, System
Dialog, scenario	At the start of each game tick, the system prints "Generate new pump at cistern? (yes/no):". If the Tester enters yes, the cistern's available pump count increases by one. The system confirms with the trace "[Cistern:<int>] New pump generated. Available pumps: <int>". The pump remains at the cistern until a Plumber picks it up using option 7. If the Tester enters no, no pump is generated.

Use-case name	**UC-18: Water Leakage and Flow Logic**
Short textual description	The system evaluates water movement through the network and registers delivered and leaked water on the scoreboard, both during each game tick and after every individual player action.
Actors	Tester, System

Dialog, scenario	Water flow is recalculated in two situations: automatically during each game tick as part of updateRoundEvents(), and immediately after every individual player action that changes network state (repair pipe, repair pump, change pump direction, connect pipe, disconnect pipe, insert pump, place new pipe, puncture pipe). The WaterFlowManager processes each spring: water travels from the spring through the network. If a pipe is PUNCTURED or has a FREE END, water cannot pass and is counted as leaked — the saboteur water counter (saboteurScore) increases by the spring's water output. If a pump is BROKEN or has no active direction set, water cannot pass and is also counted as leaked. If water successfully reaches a cistern, it is counted as delivered — the plumber water counter (plumberScore) increases. Note: plumberScore and saboteurScore represent total water volumes, not player action counts. The updated totals are displayed after each tick as "[Scores] Plumbers: <int>

Use-case name	**UC-19: End Game and Declare Winner**
Short textual description	When the timer reaches zero, the system stops the game, compares the final scores, and declares the winning team.
Actors	Tester, System
Dialog, scenario	After each game tick, the system calls checkEndCondition(). When the timer has expired (remaining time = 0), the system prints "========== GAME OVER ==========" and calls endGame(). The ScoreBoard compares delivered water (plumberScore) with leaked water (saboteurScore). If the plumber score is higher, the system prints "Winner: Plumbers". If the saboteur score is higher, the system prints "Winner: Saboteurs". If scores are equal, the system prints "Game ended in a draw."

7.3 Test plan

Name of the test-case	**TC-01: Initialize Game Successfully**
Goal	Verify UC-01: Start Game initializes with valid parameters.
Short description	The input file provides valid player names, teams (at least 2 per team), and duration. The output file should show the network created (Spring → Pipe → Pump → Pipe → Cistern), players placed on the initial pump, and the timer started.

Name of the test-case	**TC-02: Game Initialization Fails (Invalid Teams)**
Goal	Verify UC-01 correctly rejects invalid team sizes (FR-03).
Short description	The input file attempts to start a game with only 1 Plumber and 3 Saboteurs. The output file must show an error message and confirm the game session did not enter the running state.

Name of the test-case	**TC-03: Game Tick Execution**
Goal	Verify UC-02: Run Game processes a full game tick sequence.
Short description	The input script advances the game by one tick. The output file must verify that component generation was checked, water flow recalculated, turn advanced, and timer decremented.

Name of the test-case	**TC-04: Valid Move to Pipe**
Goal	Verify UC-03: Move Player allows moving to an adjacent empty pipe.
Short description	The input script issues a move command for a player on a pump to an adjacent, unoccupied pipe. The output file must show the player's position updated and the pipe marked as occupied.

Name of the test-case	**TC-05: Invalid Move (Non-Adjacent / Occupied)**
Goal	Verify UC-03 rejects illegal movements.

Short description	The input script attempts to move a player to a pipe that is either not connected to their current node OR is already occupied by another player. The output file must show an error message and the player's position unchanged.

Name of the test-case	**TC-06: Saboteur Punctures Intact Pipe**
Goal	Verify UC-04: Saboteur Punctures Pipe.
Short description	The input script places a Saboteur on an intact pipe and issues the puncture command. The output file must show the pipe's state changed to PUNCTURED.

Name of the test-case	**TC-07: Plumber Repairs Punctured Pipe**
Goal	Verify UC-05: Plumber Repairs Punctured Pipe.
Short description	The input script places a Plumber on a punctured pipe and issues the repair command. The output file must show the pipe's state restored to OK.

Name of the test-case	**TC-08: Plumber Repairs Broken Pump**
Goal	Verify UC-06: Plumber Repairs Broken Pump.
Short description	The input script places a Plumber on a broken pump and issues the repair command. The output file must show the pump's state restored to OK.

Name of the test-case	**TC-09: Plumber Changes Pump Direction**

Goal	Verify UC-07: Plumber Changes Pump Direction.
Short description	The input script has a Plumber change the active input and output of a working pump. The output file must show the pump's internal state updated to the new pipe indices.

Name of the test-case	**TC-10: Saboteur Changes Pump Direction**
Goal	Verify UC-08: Saboteur Changes Pump Direction.
Short description	The input script has a Saboteur change a working pump's routing to divert water to a dead end. The output file must show the updated directions and verify that the change was accepted.

Name of the test-case	**TC-11: Plumber Disconnects Pipe**
Goal	Verify UC-09: Plumber Disconnects a Pipe.
Short description	The input script has a Plumber disconnect an unoccupied pipe. The output file must show the pipe's state updated to FREE END.

Name of the test-case	**TC-12: Disconnect Occupied Pipe Fails**
Goal	Verify UC-09 prevents disconnecting pipes with players on them.
Short description	The input script attempts to disconnect a pipe currently occupied by another player. The output file must show an error and the pipe remains connected.

Name of the test-case	**TC-13: Plumber Connects Free End**

Goal	Verify UC-10: Plumber Connects a Pipe.
Short description	The input script directs a Plumber to connect a pipe with a FREE END to an adjacent element. The output file must show the element added to the neighbor list and the free end status removed.

Name of the test-case	**TC-14: Plumber Picks Up Pump**
Goal	Verify UC-11: Plumber Picks Up Pump at Cistern.
Short description	The input script places a Plumber at a cistern with available pumps and issues a pickup. The output file must show the Cistern's pump count decreased and the Plumber's inventory holding a pump.

Name of the test-case	**TC-15: Plumber Picks Up Pipe**
Goal	Verify UC-12: Plumber Picks Up Pipe from Cistern.
Short description	The input script places a Plumber at a cistern with available pipes and issues a pickup. The output file must show the Cistern's pipe count decreased and the Plumber carrying a pipe.

Name of the test-case	**TC-16: Plumber Inserts New Pipe**
Goal	Verify UC-13: Plumber Places New Pipe.
Short description	The input script directs a Plumber carrying a pipe to place it between a pump and a cistern. The output file must show a newly generated pipe ID correctly linked in the network topology.

Name of the test-case	**TC-17: Plumber Inserts New Pump**
Goal	Verify UC-14: Plumber Inserts New Pump.
Short description	The input script directs a Plumber carrying a pump to insert it into an existing pipe. The output file must show the original pipe removed, two new pipe halves created, and the pump connected between them.

Name of the test-case	**TC-18: System Forces Pump Breakdown**
Goal	Verify UC-15: System Randomly Breaks Pump operates deterministically.
Short description	The input script triggers the breakdown prompt with a "yes". The output file must show the first available working pump in the network changed to BROKEN.

Name of the test-case	**TC-19: System Generates Component**
Goal	Verify UC-16 & UC-17: System Generates New Pipe/Pump.
Short description	The input script accepts the generation prompts for both a pipe and a pump during a tick. The output file must show the Cistern's available counts incremented by 1 for both.

Name of the test-case	**TC-20: Flow - Successful Delivery**
Goal	Verify UC-18: Water Leakage and Flow Logic (Plumber Scoring).

Short description	With a fully intact path from Spring to Cistern, the input script executes a step. The output file must show water reaching the Cistern and the Plumbers' score increasing.

Name of the test-case	**TC-21: Flow - Leak via Puncture**
Goal	Verify UC-18: Saboteur Scoring via Puncture.
Short description	With a punctured pipe in the active path, the input script executes a step. The output file must show the water leaking to the desert and the Saboteurs' score increasing.

Name of the test-case	**TC-22: Flow - Leak via Broken Pump**
Goal	Verify UC-18: Saboteur Scoring via Broken Pump.
Short description	With a broken pump in the active path, the input script executes a step. The output file must show the flow halting at the pump and the Saboteurs' score increasing.

Name of the test-case	**TC-23: Flow - Leak via Free End**
Goal	Verify UC-18: Saboteur Scoring via Disconnection.
Short description	With a pipe disconnected on one end in the active path, the input script executes a step. The output file must show the flow escaping at the free end and the Saboteurs' score increasing.

Name of the test-case	**TC-24: Flow - Leak via Wrong Direction**
Goal	Verify UC-18: Saboteur Scoring via Rerouting.

Short description	With a pump routed to a pipe that does not lead to a Cistern, the input script executes a step. The output file must show the flow entering a dead end and the Saboteurs' score increasing.

Name of the test-case	**TC-25: Action Limit Enforcement**
Goal	Verify players cannot exceed one action per turn.
Short description	The input script issues a valid move command, followed immediately by a repair command for the same player. The output file must show that the move succeeded, but the repair was rejected due to action limits.

Name of the test-case	**TC-26: Timer Expiration & Saboteur Win**
Goal	Verify UC-19: End Game and Declare Winner.
Short description	The input script advances game ticks until the timer reaches zero with the Saboteurs holding a higher score. The output file must display "========== GAME OVER ==========" and declare "Winner: Saboteurs".

Name of the test-case	**TC-27: Timer Expiration & Plumber Win**
Goal	Verify UC-19: End Game and Declare Winner.
Short description	The input script advances game ticks until the timer reaches zero with the Plumbers holding a higher score. The output file must display "========== GAME OVER ==========" and declare "Winner: Plumbers".

7.4 Support programs for testing

At the current stage of the prototype, no separate external support program has been developed specifically for testing purposes.

Testing of the prototype is performed directly through the console-based application by executing predefined test scenarios. The prototype itself provides sufficient support for testing, as it:

- displays available actions through menu-based interaction,
- prints execution traces and system messages to the console,
- allows the tester to simulate different scenarios through user input,
- and handles conditional behavior through interactive yes/no questions.

The testing process relies on the following environment:

- Java Development Kit (JDK) version 11 or higher,
- standard command-line interface (terminal),
- the prototype program itself as the main testing tool.

Due to the interactive nature of the prototype and the scope of the current development phase, no additional converter, generator, evaluator, or automated testing utility is required. All relevant scenarios can be executed and verified manually using the built-in functionality of the system.

7.5 Tools Used

Tool	Purpose	Description
ChatGPT (AI)	Writing assistance	Used for proofreading, grammar correction, brainstorming ideas, and clarifying edge cases in the documentation.
Visual Studio Code	Development environment	Used to write, compile, debug, and organize the Java skeleton program files during development.
Draw.io	UML diagram creation	Used by some team members to create use case and communication diagrams.
PlantUML	UML diagram creation	Used by some team members to generate UML diagrams in text-based format.
Microsoft Word	Documentation	Used to write, format, and organize the Concept of Prototype document.
Discord	Communication	Used for online team meetings and coordination between team members.
Google Meet	Communication	Used for real-time collaborative coordination during the working session on April 12th.

GitHub	Version control	Used to manage, store, and share source code and documentation across the team.
Microsoft Teams	Communication	Used for file sharing and informal coordination between team members.

7.6 Protocol

Start (date & time)	**Duration (hours)**	**Performer(s) name**	**Activity description**
06.04.2026, 19:00	1.5 hours	Ibrahim, Arda, Ilgın, Hameez, Yahya, Aasif	Meeting (Discord): The team reviewed the Concept of Prototype task requirements and planned the working approach for the spring break period. Decisions: (1) Main working session scheduled for April 12th. (2) Task responsibilities delegated: Arda → 7.1.1; Aasif → 7.1.2 and 7.1.3; Ibrahim and Hameez → 7.2 and 7.5; Ilgın and Yahya → 7.3 and 7.4. Final formatting assigned to Ibrahim and Hameez. (3) All sections must be cross-verified against the skeleton source code and previous submissions before finalization.
12.04.2026, 09:00	9 hours	Arda	Individual work session: Authored Section 7.1.1 (General Description), covering the console-based execution model, input/output handling via standard streams, turn-based interaction, deterministic randomness control through yes/no prompts, and file-based testing support.
12.04.2026, 09:00	9 hours	Aasif	Individual work session: Authored Section 7.1.2 (Input Language) and Section 7.1.3 (Output Language). Defined all textual inputs and outputs of the prototype across thirteen output subsections. All entries verified for exact string compliance against the Java source code.

12.04.2026, 09:00	9 hours	Ibrahim, Hameez	Collaborative work session (Google Meet): Jointly authored Section 7.2 (Real Use-Cases), defining 19 use-case tables covering all game behaviors from Start Game to End Game. Each use case verified against the skeleton source code for exact alignment with method behavior and console output strings. Also responsible for final formatting and consistency checking of the complete document.
12.04.2026, 09:00	9 hours	Ilgın, Yahya	Collaborative work session (Google Meet): Jointly authored Section 7.3 (Test Plan) and Section 7.4 (Support Programs for Testing). Test cases cross-referenced against Section 7.2 use cases and the input/output language in Section 7.1.